THE UNIVERSITY OF QUEENSLAND

AUSTRALIA

Detecting insecure weak memory behaviour in ARM64 binary code

by
Andrew Brown

School of Electrical Engineering and Computer Science,
The University of Queensland.

Submitted for the degree of
Master of Cyber Security

October 2025

Abstract

Modern computing hardware, in the pursuit of higher speeds and further efficiency, no longer guarantees sequential execution of instructions nor sequentially consistent memory effects. These effects are summarised by the notion of a *weak memory model*, describing how and when instructions may execute out-of-order, or their side effects may become visible out of order. This compromise the soundness of traditional reasoning methods for concurrent code, potentially leading to incorrect behaviour and security bugs.

In this thesis, I have worked towards implementing a contemporary approach, "Reordering Interference Freedom," as a practical tool, in the form of Oregano. This tool takes input from an off-the-shelf binary disassembler, as well as a concurrent specification provided by the programmer. With these, the tool is able to "lift" the disassembly, automatically identify all instructions that may execute out-of-order, and engage an automatic theorem prover to assess whether these instructions may affect the program's concurrent behaviour.

Contents

Abstract **1**

List of Figures **3**

1 Introduction **4**

1.1 Confidentiality 4

1.2 Existing Techniques 5

1.2.1 Weakest Precondition Reasoning 5

1.2.2 Model Checking 7

1.3 Binary Analysis 9

1.4 Weak Memory 10

1.4.1 ARMv8 Architecture 11

2 Current Approaches **13**

2.1 Model Checking 13

2.2 Reasoning 14

2.2.1 Reordering Interference Freedom (RIF) 14

2.3 Automation of RIF 16

2.3.1 Combinations 17

3 The Oregano Tool **19**

3.1 Compilation 20

3.2 Lifting 21

3.3 Pair Generation 22

3.4 Sequential Solver Encoding 24

3.4.1 Symbolic Memory 26

3.5 Combinations & Concurrent Solver Encoding 27

3.6 Evaluation 28

4 Conclusion **30**

4.1 Future Work 30

4.2 Final Remarks 31

List of Figures

1.1 Motivating example: Secret Write . . . 5
1.2 Secret Write with additional *wpif* assertions added. . . . 6
1.3 Example rely/guarantee specification for *Secret Write* . . . 7
1.4 Sample control-flow model for Secret Write . . . 8
1.5 Example compiler dead-code elimination . . . 9
1.6 Secret Write with illustrated weak-memory behaviour . . . 11

2.1 Proof-implication style presentation of RIF . . . 15
2.2 Behavioural-subset style presentation of RIF . . . 16

3.1 The OREGANO architecture and submodules . . . 19
3.2 Original C code for *minimal.c* reordering example. . . . 20
3.3 Compiled assembly code for *minimal.c* reordering example. . . . 21
3.4 ASL fragments for instructions 7 and 9 of *minimal.c* reordering example . . . 22
3.5 Semantic summary for instructions 7 and 9 of *minimal.c* reordering example . . . 23
3.6 "Constraint flow" over program state transitions for a RIF-check with no specification. . . 25
3.7 Partial SyGuS-language solver code for instructions 7 and 9 of *minimal.c* . . . 26
3.8 "Constraint flow" over program state transitions for a RIF-check with specification. . . . 27
3.9 Timing results for lifting + pair-generation for various functions . . . 29

Chapter 1

Introduction

Providing security guarantees for concurrent programs by means of formal verification is highly desirable. This task is already difficult, but becomes further complicated when the processor executing a program performs microarchitectural optimisations. These optimisations, or "weak memory" behaviour, further complicate the task of proving correctness.

In this chapter, we will define "confidentiality," a desired security property for any code, then build up to and examine existing techniques for proving that binary executables uphold this requirement. We will also briefly define "weak memory" behaviour.

1.1 Confidentiality

In developing a fundamental mathematical theory underpinning cybersecurity-focused verification, properties of programs have been categorised as either safety: informally, "bad things don't happen ever", or liveness: informally, "good things will happen eventually" [1].

We will focus on the verification of "information flow security", or, again informally stated, "programs do not leak secret information to un-authorised users". This property is a safety property, where our "bad thing" that must not occur is the leaking of secret information. Our formal basis for this property is described by Sablefeld and Myers [2]: "Information controlled by a confidentiality policy cannot flow to a location where that policy is violated."

Presented in Fig. 1.1 is an example snippet of C-style pseudocode serving as a prototypical example that we might like to prove secure, known as *Secret Write*. This program appears to be correct, in its current form. When *Thread 1* executes, it begins by taking the `lock`, which remains held for the entire duration that `shared` contains our `secret` value. Immediately prior to the releasing of this lock, `shared` is zero'ed out with a constant value. Any other thread executing code – including *Thread 2* – is able to use the `lock` variable for mutual exclusion, only proceeding when it indicates that `shared` is safe to read.

Thread 1

```
take_lock(lock);
shared = secret;
...
shared = 0;
release_lock(lock);
```

Thread 2

```
take_lock(lock);
printf(shared);
release_lock(lock);
```

Figure 1.1: Motivating example: Secret Write

1.2 Existing Techniques

1.2.1 Weakest Precondition Reasoning

Firstly, we will examine the logical approach to verifying information flow security. Our target is the $wpif_{\mathcal{RG}}$ logic, published by Winter et al. [3]. This logic is based around the use of a weakest precondition predicate transformer, which is used on all instructions in the program. Essentially, at each step, the transformer $wpif_{\mathcal{RG}}$ takes in a postcondition Q and an instruction, and computes the "weakest precondition" required for the instruction to reach a state that satisfies Q but also upholds the confidentiality of the program. This improves on earlier work by the same group [4] by utilising backwards-directed reasoning techniques, as opposed to forwards-directed. This greatly simplifies the proof by ensuring that only necessary information is carried through the reasoning process. Intuitively, strongest- postcondition reasoning does not "know" which state might be relevant – it must work with the entire state at all program steps.

When moving to concurrent code, both forms of reasoning can be augmented with rely/guarantee conditions [5]. Relies and guarantees are a form of abstract, formal summaries of the behaviour of each thread, in terms of what behaviour the thread expects the environment to maintain at all times (relies) and what behaviour the thread guarantees the environment can expect at all times (guarantees). Rely/guarantee reasoning is a widely used modern technique for compositional reasoning about concurrent code, enabling every thread to be considered and reasoned about locally, in total isolation from other concurrent components [5]. The $wpif_{\mathcal{RG}}$ logic of [3] utilises rely-guarantee judgements in this way to ensure concurrent code remains verifiable.

As an example of how this reasoning process may occur, we will apply the $wpif_{\mathcal{RG}}$ logic to *Secret Write*. Firstly, the system will automatically glue in the assertions to construct the program(s) seen in Fig. 1.2 – assuming we have the ability to assert that `"<Shared is not secret>"` in predicate form. Notably, this program is largely unchanged, as the only sequential condition that must be true within an individual thread is that our `shared` variable must not contain a `secret` when it is printed to the user.

In the general case, the verification conditions added to the reasoning are much more complex. The

Thread 1

```
take_lock(lock);
shared = secret;
...
shared = 0;
release_lock(lock);
```

Thread 2

```
take_lock(lock);
assert("<Shared is not secret>");
printf(shared);
release_lock(lock);
```

Figure 1.2: Secret Write with additional *wpif* assertions added.

$wpif_{\mathcal{RG}}$ logic checks two major cases:

1. For every assignment, a variable's security classification, $\mathcal{L}(x)$, must not exceed the security level, $\Gamma_E(e)$, of the expression assigned to it.
2. For every branch, including both conditionals and loops, the guard of the branch does not depend on secret data.

There is an additional case for value-dependent security classifications – where $\mathcal{L}(x)$ may depend on another variable – but that is out-of-scope for this work. If all of these conditions are met at all relevant locations, then we are able to prove our program does not leak secret information.

Rely-Guarantee Reasoning

Following this process, we now have a proof that our code, executed sequentially, upholds confidentiality. Subsequently, the $wpif_{\mathcal{RG}}$ logic engages rely/guarantee reasoning in order to prove that the programs are correct under concurrent execution. This invoves the programmer manually specifying rely and guarantee conditions for the concurrent execution of each thread, which we can now demonstrate.

In lieu of delving into the lower-level behaviour of our `lock` at this stage, we will assume the existence of a series of functions, satisfying:

$$\forall v \in \mathcal{V}ars, i \in \mathbb{N} \quad \texttt{held}_i(v) \implies \forall j \in \mathbb{N} \,.\, j \neq i, \neg\texttt{held}_j(v) \tag{1.1}$$

Or, in other words, for any lock-variable v, if a thread i holds v, no other thread may also hold the lock. Given `held` and the security level Γ of all variables at all points in the program (from the $wpif_{\mathcal{RG}}$ logic), we may provide our rely/guarantee specification as in Fig. 1.3. Conventionally, these specifications are provided as predicates over two states, meaning we have both `shared` and `shared`$'$, and `lock` and `lock`$'$. In this case, the prime variables describe the next state that we move to from the original non-prime variables.

There are many possible formulations of rely and guarantee conditions for this example. In this case, we have specified that if Thread 1 holds the `lock`, the environment must not change anything, and it will uphold that `shared` holding a secret value means it must have previously taken the `lock`. Additionally,

Thread 1

Rely:

$$\texttt{held}_1(\texttt{lock}) \implies \texttt{held}_1(\texttt{lock}') \wedge \texttt{shared} = \texttt{shared}'$$

Guar: $\Gamma(\texttt{shared}) \implies \texttt{held}_1(\texttt{lock})$

Thread 2

Rely:

$$\texttt{held}_2(\texttt{lock}) \implies \texttt{held}_2(\texttt{lock}') \wedge \Gamma(\texttt{shared})$$

Guar: $\texttt{shared} = \texttt{shared}'$

Figure 1.3: Example rely/guarantee specification for *Secret Write*

Thread 2 has the slightly changed environmental condition that if it holds the `lock`, `shared` must not contain a `secret`, but upholds the much stronger guarantee condition that it does not change the `shared` variable at all.

In order to actually prove the two threads preserve this behaviour, the $wpif_{\mathcal{RG}}$ logic inserts the rely/guarantee specification, as calls and assertions, into the ordinary sequential code. Essentially, the rely is converted into a function summarising the behaviour of the environment, called inbetween every ordinary sequential code step, and the guarantee is converted into an assertion, where the state before every ordinary sequential code step is recorded and compared to the state afterwards [5], [6]. For an arbitrary instruction α, operating on some program state σ, we would therefore convert the program from left-to-right, as shown in Eq. (1.2).

$$\begin{aligned} \sigma = \alpha(\sigma); \qquad & \sigma = \mathcal{R}(\sigma); \\ & \sigma' = \alpha(\sigma); \\ & \texttt{assert}(\mathcal{G}(\sigma, \sigma')); \\ & \sigma = \sigma'; \end{aligned} \tag{1.2}$$

Note we must track our state σ and σ' at all times. We begin where they are the same, then we update our σ' based on the executed instruction α, then our $\mathcal{G}$ forms a constraint over both states, before we reset them back to the same again before the next instruction.

This clearly results in a much longer sequential program to verify, so we will not manually do so for our *Secret Write* example. Hopefully, it remains clear that the rely/guarantee reasoning for this example should successfully prove the code to be correct. The important takeaway from this example is that rely/guarantee reasoning is able to break down the problem of concurrent verification into simply longer-form sequential verification, given a human-generated rely/guarantee specification.

1.2.2 Model Checking

Another prominent technique is model checking, including its modern extension, stateless model checking [7], [8]. This technique involves forming a finite-state model of the program under consideration, then,

conducting a complete state-space search over the program to determine whether failure states are reachable.

Stateless model checking specifically refers to the technique of abstracting over the state so as not to require tracking the values of all program variables at all points during the search [8]. Model checking is generally highly automatic, as programs can construct these models and find equivalences between them without programmer input. However, what they gain in automatability, they lose in compositionality. This results in the "state-explosion problem" as tools must handle exponentially more possible behaviours over more and more threads.

To prove *Secret Write* is secure, model-checking style, we must construct a finite-state model of our program. This takes the form of a directed graph, provided in Fig. 1.4. For presentation, the text in this diagram has been heavily abbreviated.

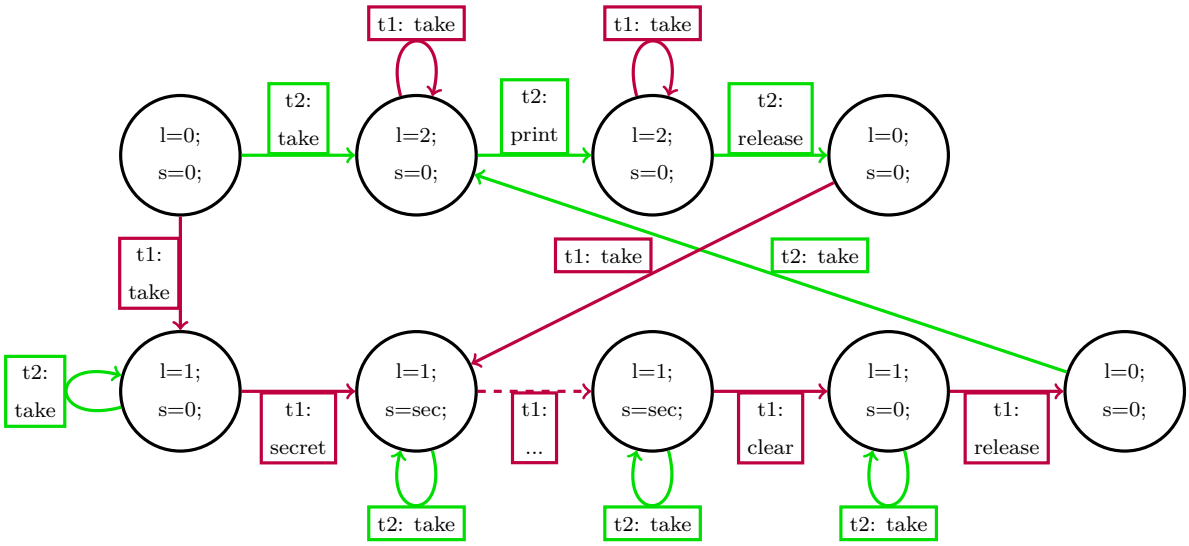

Figure 1.4: Sample control-flow model for Secret Write

Once we have this finite-state model, it then suffices for us to search the graph for any behaviour that may break our confidentiality specification. In this model, much as in Fig. 1.2, the only problematic case we must investigate is whether we could possibly execute a transition over `printf`(`shared`), coming from a state node where `shared` contains `secret` data. In this case, the program can be proven secure by simply following the edges to determine that at the only print, the most recent write to `shared` was of a constant, therefore non-secret, value. Research in the model checking space focuses on developing faster algorithms for performing this search [7], as well as further equivalences between different finite-state models [9], [10], reducing the state space that must be checked.

In general, between the model-checking and the reasoning approaches in the literature, the choice is between compositional techniques requiring a human to devise a sufficient mathematical specification for the program, or model-checking requiring significant execution time to automatically explore all behaviour of a concurrent program – potentially even timing out.

1.3 Binary Analysis

As further background, we will also discuss the choice to work on "low-level" binary code. Verification of source code is well-established to be potentially fraught with danger [11]. Compilers of source code into binary code undertake optimisations that may affect the security properties of a program. A common example is so-called "dead-store elimination," in which a write of a variable that is never used again can be eliminated. For programs written in the C language, in which allocated memory is returned to the operating system via a call to the function `free()`, vulnerabilities of the form in Fig. 1.5 is commonly uncovered [12]

Source Code

```
0  shared = secret;
1  ...
2  shared = 0;
3  free(shared);
```

Compiled Binary

```
0  mov  x1, 0xDEADBEEF // secret data
1  str  x1, [x0]
2  ...
3  call 0x1000         // free
```

Figure 1.5: Example compiler dead-code elimination

In our compiled binary, we are assuming that the address of the `shared` variable is already loaded into register `x0`. The remaining code moves some secret data into register `x1`, and performs a store of this data to our previously-loaded address.

However, the compiler determines that line 3 – zero'ing out our `shared` variable – is an inefficient use of our CPU, as the stored value is never again read by the program; it is immediately returned to the system. It eliminates this "dead" store, leaving us with only the call to `free()`[1]. This may be perfectly valid in some code, but in other code, the security model may be insufficient to allow this behaviour to occur. The value available in `shared` will be released back to system memory and could be incidentally picked up, read, and disclosed by this or another program.

In this specific case, the vulnerability is effectively solved, as source code static analysis tools are able to detect and complain about this common problematic pattern. Furthermore, following the publicisation of this issue, modern compilers all offer some form of explicit zero'ing function that will never be optimised out of the generated code [12]. However, binary analysis tools are able to detect this vulnerability without any specifically-crafted rules – a binary analyser would only see the right-hand-side version of this code, where the problem is detectable by $wpif_{\mathcal{RG}}$ without any prior knowledge of the particular antipattern. In this way, binary analysis tools lose some additional information and utility for a programmer, who is used to working with source code, in order to gain assurance that their verification efforts definitely hold for the final artifact that is executed.

[1] Note that conventionally in compiled binary, function arguments are passed in registers automatically, so the call to `free()` takes `shared` as a parameter from register `x0` without needing a specific parameter list, as source code would.

The choice to travel down to binary level provides us with another benefit: for many functions, especially smaller ones, we no longer need to break up the code and provide different pre- and post-conditions for different functions. Compiler inlining [13] can result in these smaller functions disappearing from the compiled code. For example, we may anticipate that the mysterious `take_lock()` and `release_lock()` functions disappear from *Secret Write*, thus absolving us of a need to specify their behaviour.

A final reason to examine our code at a binary level is that describing a weak memory model is substantially easier when working at the level of the underlying hardware [14], as source code level statements and expressions can often combine many memory accesses into one construct. This aspect will be discussed further in Section 2.2.1.

1.4 Weak Memory

We have introduced current approaches to verifying the correctness, or confidentiality-preservation, of computer programs. Our final piece of background is then to meet the mechanical enemy standing in our path. The basic premise of weak memory is simple: due to caching, pipelining, or buffering, the visible memory behaviour of a thread may not be what a reader would immediately suspect based on just the written code [15]. At a binary level, this behaviour comes from the CPU itself, which may execute instructions out-of-order to better fill its internal "pipeline" of instructions under execution [16], [17], [18].

A generalised notion of weak memory can therefore be provided in terms of reordered instructions: the processor may execute some future instruction β before the current instruction α, so long as the instructions are independent of each another [16], [19]. On the surface, this is a reasonable behaviour, and enables significant processing speedups, as the different subsystems of a processor can be tuned to ensure that all of them remain busy as much as possible.

However, the requirement of simple independence is insufficient to ensure that reordered code upholds the guarantees of the program in the presence of cross-core concurrency. In other words, the processor is unaware of the wider context in which its instructions are executing, and may break assumptions that this context makes. Executing our *Secret Write* example from earlier reveals this problem, as in the presence of weak memory, key instructions may reorder:

At the beginning of each thread, we assume that `x1` holds the address of the lock, and `x0` holds the address of the shared variable, though note that these registers do not have to be the same in each thread. The highlighted pair of instructions perform the clearing of `shared` and the releasing of `lock`, respectively, but note that the compiler here has produced code where each instruction is independent, using different registers, and therefore those two lines could be reordered. This would clearly break the security guarantees of the program, as we would now be unlocking the `lock` before clearing `shared`, providing a small but nevertheless not impossible window for *Thread 2* to execute and print our `secret` data.

Thread 1

```
0 mov  x2, 0;
1 cas  x2, 1, [x1]; // atomic spinlock
2 cbz  x2, 0x1;     // retry on failure
3 str  secret, [x0]; // store secret
4 ...
5 mov  x2, 0;
6 str  x2, [x0];     // clear shared
7 mov  x3, 0;
8 str  x3, [x1];     // unlock
```

Thread 2

```
0 mov  x2, 0;
1 cas  x2, 1, [x1]; // atomic spinlock
2 cbz  x2, 0x1;     // retry on failure
3 ldr  x0, [x0];    // load shared
4 call 0x1000       // printf(x0);
5 mov  x3, 0;
6 str  x3, [x1];    // unlock
```

Figure 1.6: Secret Write with illustrated weak-memory behaviour

This may be a result of any number of microarchitectural optimisations – in the ARMv8 architecture, as discussed below, this would be a form of "Reordered Access". While, within *Thread 1*, this is semantically equivalent, the processor is unaware that our wider program's security is predicated on the assumption that these statements to execute in order. Any tool purporting to prove that our *Secret Write* example is correct may not detect this vulnerability if weak memory behaviour is not considered – a critical flaw.

1.4.1 ARMv8 Architecture

Evidently, the decision of whether or not weak memory behaviour is possible is highly dependent on the manufacturer of the chip in question. In this project, we consider only ARMv8. Primarily, this is due to the degree of existing tool and specification support provided, compared with other processor architectures, though it is also motivated by the fact that ARMv8 is far more permissive about the weak memory behavior the system may exhibit than other common architectures, including Intel's `x86_64`.

As an example of this support, ARM Limited have released documentation explaining exactly the types of weak memory behaviour programmers should expect from its hardware [18]. In this document, three types of weak memory behaviour are permitted:

- Merged Access: the processor may detect and merge multiple accesses, even of consecutive locations, into a single access.
- Reordered Access: the processor may reorder accesses, so long as they target different locations.
- Speculative Access: the processor may read memory without being specifically asked to by the executing code.

Of these three, we primarily care about Reordered Access – this provides us with the out-of-order behaviour that causes problems as described above. It is important to also consider Merged Access, as

this still represents altering the behaviour of the code under execution, though it is much less likely to cause problems with the concurrent behaviour of the code. Speculative Access is out of scope, however. Processors deciding to undertake speculative execution is based on a far more complex system of machine learning [20] that cannot be easily axiomatised. It is therefore not considered by the logic underpinning this project [14], as well as the original rely/guarantee work providing context for that logic [3].

Chapter 2

Current Approaches

We now understand the preliminaries of formal verification for our desired security property, as well as the problem posed if our source-code-level verified code exhibits reordering, or other weak memory behaviour, when under actual execution.

In this chapter, we will examine various tools and methods proposed in the current literature to manage the weak memory problem, including Reordering Interference Freedom (RIF), the primary approach used in this project. We will end by proposing a method of expanding automation for RIF, with the goal of reducing the degree of human intervention required for a proof.

2.1 Model Checking

The first contemporary approach to dealing with the weak memory problem is to extend current model checking work. In some ways, this is a natural choice – model checkers already have to incorporate a level of nondeterministic choice to handle inter-thread concurrency, so merely introducing one more level to handle weak-memory-based intra-thread concurrency is relatively accessible. Multiple groups have worked on this foundation [10], [21] to develop model checkers for different weak memory architectures.

The first of those was the work by Abdulla et al. [10]. This approach focuses on the abstract Total Store Order (TSO) and Partial Store Order (PSO) memory models. TSO describes a system in which each thread, and for PSO, each shared variable in each thread, have their own write buffers [22]. At some future point in the program's execution, any or all buffers may be flushed, and the sum total of all writes they hold will instantly become visible to all other threads. Notably, the x86-64 instruction set in most modern consumer computers implements the TSO memory model [22].

The [10] approach involves a novel representation of program states, which they call *chronological traces*. This approach provides them with a new partial-order relation on program states which they extend to create a new series of equivalence classes for weak-memory behaviour. Their technique is optimal for their desired memory models, in the sense that for programs that do not exhibit any weak-memory behavior, they are able to emit only as many classes to be model-checked as a sequentially-consistent memory model would [22].

Similar is the approach of Kokologiannakis and Vafeiadis [21], which also utilises model checking, but

uses an alternative theoretical background from [9]. This alternative theoretical background focuses on what they call "dependency-tracking" memory models. These can express significantly more complex models than TSO or PSO, including that of ARMv8. The name comes from the fact that these memory models are able to reorder instructions satisfying some check against dependencies, which, as we saw in Section 1.4, is a sufficient approach to axiomatise all memory models. This approach entails tracking dependencies across assignments during the state-space search. These dependencies can then be used to decide which instruction should be executed next. Furthermore, their approach is parameterised by the memory model, enabling their GenMC tool to handle other memory models, provided they can be described in the tool's format.

Optimisations championed by [21] include detection and reduction of equivalence classes based on lock-taking behaviour, reducing program execution traces where one thread holds a lock and others unsuccessfully attempt to acquire it [23], or detection of scenarios where threads executing identical code may generate different traces modulo only the thread number that exhibits each behaviour [24].

The GenMC tool demonstrates a model checking approach that is is flexible and automatic, and covers even the complex weak-memory behaviour exhibited by ARMv8. However, in practice, model checking still struggles with scalability, requiring upwards of 4GB of memory [21] in order to complete checking of a program using musl libc's [25] mutex implementation.

2.2 Reasoning

In the other direction lie compositional-reasoning approaches. These invariably incorporate some form of rely/guarantee reasoning, as described in Section 1.2.1, to handle the program's concurrent behaviour without running into state-space-explosion issues. The Owicki-Gries Release/Acquire (OGRA) logic, [26], developed out of the same research group as [21], provides a system wherein the weak memory behaviour is baked into the mathematical framework directly.

The core of OGRA is based on Owicki-Gries reasoning [27], a precursor to rely/guarantee work [5]. It takes the form of a stronger system wherein data-race behaviour is prohibited by a condition threads must not interfere with each other. Essentially, all assertions in one thread must be still hold over all assignments in all other threads. OGRA extends this condition even further to handle weak memory behaviour by forgoing the visibility of states between threads at each assignment [26]. This approach enables slightly more abstraction over different threads than a model checking approach, however, it is highly complex, and still not truly thread-local. Baking weak memory behaviour into the same proof as the general correctness of the program results in complex rules that are difficult to automate and difficult for programmers to utilise effectively.

2.2.1 Reordering Interference Freedom (RIF)

Going further, the RIF approach developed in [14] enables truly compositional splitting of the reasoning task. Unlike the approach developed by OGRA [26], RIF provides the benefits of a compositional

approach without deeply modifying the original program logic under consideration. The theory of RIF is built on the ability to detect all instruction pairs that may reorder. If every reorderable pair can be reordered without affecting, or being affected by, the program's concurrent rely/guarantee specification, then the program can be proven correct, even in the face of weak memory behaviour.

This provides another core reason for us to be working with binary code – this is the level that memory model behaviour is defined on. By performing our weak memory reasoning on this level, we are able to utilise information from the designers of the memory model itself. In this case, we are able to use the definitions provided by ARM Limited themselves [18].

To summarise the formalisation of RIF, we begin with a general rely/guarantee judgement for "atomic" instructions:

$$\mathcal{R}, \mathcal{G} \vdash_{\mathsf{a}} P\{\alpha\}Q \tag{2.1}$$

in which an instruction α starting in the state P will terminate in state Q, subject to some context $\mathcal{R}$ and additionally upholding all conditions from $\mathcal{G}$. Proving this requires showing that the pre- and post-conditions are stable under $\mathcal{R}$ (i.e. the state performing some action will not invalidate the pre- or post-conditions), that the instruction satisfies the guarantee $\mathcal{G}$, and that the ordinary serial Hoare triple $P\{\alpha\}Q$ holds. From here, we can generate rely/guarantee judgments for a sequentially-consistent "component" using rules defined inductively over composition of instructions – sequential composition, nondeterministic choice, iteration, and parallel composition [14]. These rules will not be re-presented here, but they operate in a fairly standard manner.

As a point of notation, $\beta_{\langle\alpha\rangle}$ is the instruction β having potentially been modified by "forwarding" over α. Forwarding is a behaviour occurring from a combination of "Reordered Access" and "Merged Access" – an instruction pair `a := b; c := a;` may have an access ignored due to merging, becoming `a := b; c := b;`, and noting that these instructions are now independent, they may be reordered to `c := b; a := b;`. In this example, the instruction `c := b;` is not just reordered, it is also fundamentally altered, and this is what $\beta_{\langle\alpha\rangle}$ captures. In the event of a non-forwarding reorder, $\beta_{\langle\alpha\rangle}$ is just β, as the instruction is not altered by a normal reordering. This behaviour requires special attention.

Component judgment in hand, RIF enables us to provide a non-sequential-consistency-assuming proof by adding in a check for RIF, as in Eq. (2.2), which is defined, again, starting with an "atomic" check over one pair of reorderable instructions.

$$\begin{aligned} rif_{\mathsf{a}}(\mathcal{R}, \mathcal{G}, \alpha, \beta) \mathrel{\widehat{=}} \forall P, Q, M.\ & \mathcal{R}, \mathcal{G} \vdash_{\mathsf{a}} P\{\alpha\}M \wedge \mathcal{R}, \mathcal{G} \vdash_{\mathsf{a}} M\{\beta\}Q \\ \implies \exists M'.\ & \mathcal{R}, \mathcal{G} \vdash_{\mathsf{a}} P\{\beta_{\langle\alpha\rangle}\}M' \wedge \mathcal{R}, \mathcal{G} \vdash_{\mathsf{a}} M'\{\alpha\}Q \end{aligned} \tag{2.2}$$

Figure 2.1: Proof-implication style presentation of RIF

This defines atomic-RIF for a rely $\mathcal{R}$, guarantee $\mathcal{G}$, and two reorderable instructions β and α as an implication that if α executes, taking us from *any* pre-state P to *any* intermediate state M, and β

executes taking us from M to *any* Q, then there must exist an alternative middle-state M' such that $\beta_{\langle\alpha\rangle}$ takes us from P to M', and α takes us from M' to Q.

Or, in briefer language, for all pre- and post-states P, Q, if we can prove the program executes in-order via some middle-state M, we must be able to prove that there exists a middle-state M' such that the out-of-order execution still travels from the same P to the same Q.

The full-program RIF check is then constructed out of this atomic check by simply performing the check for all reorderable pairs of instructions within the program. Formally in [14], this is defined based on sequentially-composed prefixes of all instructions, but there is no immediate need to rely on this construction; any method of determining the reorderable pairs will suffice.

The paper [14] also provides an Isabelle/HOL encoding and soundness proof. In terms of its utility relative to other approaches, it is hopefully clear that the nature of RIF as a separate check to the core rely/guarantee reasoning is immensely valuable. By splitting these checks, the effects of weak memory can be ignored, and the program verified using other, existing approaches to R/G reasoning.

2.3 Automation of RIF

RIF as formulated in the original paper [14] may be our desired approach, but we will slightly reformulate it for the purposes of our solver. Firstly, an implication between rely/guarantee judgements is undesirable, as it is difficult to encode these in a way that an automated solver tool will be able to effectively use. We therefore have relied on an alternative, formulation that is more related to the equivalent notion [5] of behavioural subsets. In essence, rather than stating "a proof in-order implies a proof out-of-order", we state that all possible behaviour visible out-of-order could still occur in-order.

Our behavioural-subset formulation is stated in terms of two sequential programs, constructed from the code as well as by following Eq. (1.2) to sequentialise the rely specification. We also borrow the notion of the "strongest-postcondition" sp, to give us the set of states that the program may end in:

$$rif_{\mathsf{a}}(\mathcal{R}, \mathcal{G}, \alpha, \beta) \mathrel{\widehat{=}} \forall P :$$

$$\begin{array}{lll}
\exists C_1 \mathrel{\widehat{=}} \{ & \exists C_2 \mathrel{\widehat{=}} \{ & \\
\mathcal{R}; & \mathcal{R}; & sp(P, C_2) \subseteq sp(P, C_1) \\
\alpha; & \beta_{\langle\alpha\rangle}; & \\
\mathcal{R}; & \mathcal{R}; & \\
\beta; & \alpha; & \\
\mathcal{R}; & \mathcal{R}; & \\
\} & \} &
\end{array}$$

Figure 2.2: Behavioural-subset style presentation of RIF

This can be read as stating that for two "mini-programs" – C_1 representing in-order execution and C_2

representing out-of-order execution – we must show that the set of all behaviours exhibited by C_2 is a subset of the behaviours exhibited by C_1. This represents the same real property as the original proof-implication style of RIF, but is substantially easier to encode in an automated solver. Note also the reason why are using strongest-postcondition style transformation here, as opposed to weakest-precondition; this choice means our originally quantified state is the pre-state, which we can more easily make guesses about based on the specification.

As another point of nuance, our guarantee conditions are also treated specially by this system. The reason for this is that our state is expressed as simply a series of quantified variables in the solver, with no possibility for a program-level error to occur. Errors such as division by zero are lifted to occur at an actual solver level; we do not have a way to encode a state like "the program has failed" without a corresponding solver failure. This means our guarantee conditions cannot be flatly inserted into these programs as `asserts` in the same way as Eq. (1.2). They are instead expressed as additional solver constraints on the intermediate states immediately after $\beta_{\langle\alpha\rangle}$ or α in the reordered mini-program. Further detail on this aspect of the encoding is provided with examples in Section 3.4.

2.3.1 Combinations

In the current state, we will doubtless present many false positives to a user, as we simply do not know anything about the initial state we find ourselves executing from. This is visible already in Fig. 1.6: we do not know which address registers alias which program variables; we do not know the contents of the value registers; and we do not know the starting state of our program variables.

If we consider only the reordered pair in isolation, which is a core requirement of the RIF approach, we must handle the cases where our `lock` is already unlocked, or our registers actually alias the opposite program variables (unlocking lock before we clear shared even in the normally-ordered case), or even whether the value we are storing to `shared` is a non-secret value at all.

Existing RIF work [14] resolves this problem by requiring the programmer to manually provide preconditions for each pair, specifying in what sort of state the pair could possibly execute. Our goal is to improve on this process, so we must come up with an automatable way to resolve these pre-state ambiguities.

Fortunately, with a specialised form for our code's specification, we can utilise it to partition our initial state space into different relevant groups of states, and check these "combinations" over the in-order code as a first filtering step. With a specification provided in terms of:

- variables $\in \mathcal{V}ar$,
- uninterpreted boolean functions $\mathcal{V}ar \to Bool$, or
- conjunctions, disjunctions, and other operations thereof,

we can start by asserting in our pre-state that our specified functions of variables are either true, or false.

To demonstrate what this means, the specification for *Secret Write* may be provided in terms of two uninterpreted functions, a `secret` function representing Γ, and a `held` function representing $\texttt{held}_i$ (for

i being the thread number under investigation). Recalling our guarantee for *Secret Write*'s first thread, it is quite natural to provide a specification in terms of these uninterpreted functions:

$$\texttt{secret}(\texttt{shared}) \implies \texttt{held}(\texttt{lock}) \tag{2.3}$$

From just this specification, our tool can now automatically infer that the relevant descriptions for all variables in the state are simply whether or not they are `secret`, and whether or not they are `held`. We care not what these functions actually mean; we only care whether our variables meet or do not meet their criteria.

With these, the combinations for the reordered pair from Fig. 1.6 would be:

- $\texttt{secret}(\texttt{shared}) \lor \neg\texttt{secret}(\texttt{shared})$
- $\texttt{held}(\texttt{lock}) \lor \neg\texttt{held}(\texttt{lock})$
- $\big(\texttt{secret}(\texttt{x2}) \lor \neg\texttt{secret}(\texttt{x2})\big) \land \big(\texttt{held}(\texttt{x2}) \lor \neg\texttt{held}(\texttt{x2})\big)$
- $\big(\texttt{secret}(\texttt{x3}) \lor \neg\texttt{secret}(\texttt{x3})\big) \land \big(\texttt{held}(\texttt{x3}) \lor \neg\texttt{held}(\texttt{x3})\big)$
- $[\texttt{x0}] = \texttt{shared} \lor [\texttt{x0}] = \texttt{lock}$
- $[\texttt{x1}] = \texttt{shared} \lor [\texttt{x1}] = \texttt{lock}$

As an aside, we will define here the *canonical combination* as the initial state conditions that the code may actually encounter at runtime. For example, the *canonical combination* for the stores shown in Fig. 1.6 would be:

$$\texttt{secret}(\texttt{shared}) \land \texttt{held}(\texttt{lock}) \land \neg\texttt{secret}(\texttt{x2}) \land \neg\texttt{held}(\texttt{x3}) \land [\texttt{x0}] = \texttt{shared} \land [\texttt{x1}] = \texttt{lock}$$

Having generated our combinations, of which there will likely be many, we can return to the original premise of RIF to help us cut them down. Recall that the notion of RIF is to prove that the code is correct with respect to reordering, but *assuming it is correct in-order already*. This fact is crucial for cutting down our combinations: we can simply perform a preliminary check with the solver to test whether our initial conditions will uphold the guarantees required by in-order execution.

Alternatively, further optimisations can be made to the number of combinations to be checked: the CPU already guarantees that the instructions are independent, so we do not need to check any combination where `[x0]` and `[x1]` reference the same variable. Static analysis passes could cut this requirement down further; this possibility is discussed in Section 4.1.

Optimisations aside, what our tool is required to do, given a pair and a series of combinations, is to first construct the intermediate states that would be encountered by in-order execution, constrained by the guarantees of the program. If this process fails, we may throw away the combination, assuming that it was simply incorrect. After this filtering process, only valid in-order combinations will proceed to the actual RIF check in Fig. 2.2, to then be checked again and determine whether out-of-order execution may change the behaviour of the program.

Chapter 3

The Oregano Tool

As has been alluded to, as part of this project, substantial effort went into the creation of an automated tool for RIF checks, or rather, our automatability-focused reformulation discussed above. The result of this is the Oregano tool (*Overestimating Reordering, Execution Guarantees Are NOrmal*), the architecture of which is described in Fig. 3.1.

In short, the tool takes in compiled-and-lifted binary code, then the lifter module described in Section 3.2 uses ASLp to lift the binary further into an intermediate language. The next module, Section 3.3, uses Datalog to detect reorderable pairs of instructions. We then discuss the Solver in two stages – first, Section 3.4 discusses the basic nature of the encoding without reference to the concurrent specification, then Section 3.5 discusses extending this encoding with the specification and its "combinations" as we introduced earlier.

Throughout this module-by-module breakdown, we will refer back to a minimal example with no interesting behaviour, shown over the page as Fig. 3.2. This example exhibits reordering between two stores to x and y, and could concievably have a rely/guarantee spec governing the use of these global variables. The benefit of this example is that there is no control-flow or complex code behaviour, as our original *Secret Write* had, hopefully making the links betwen each step clearer.

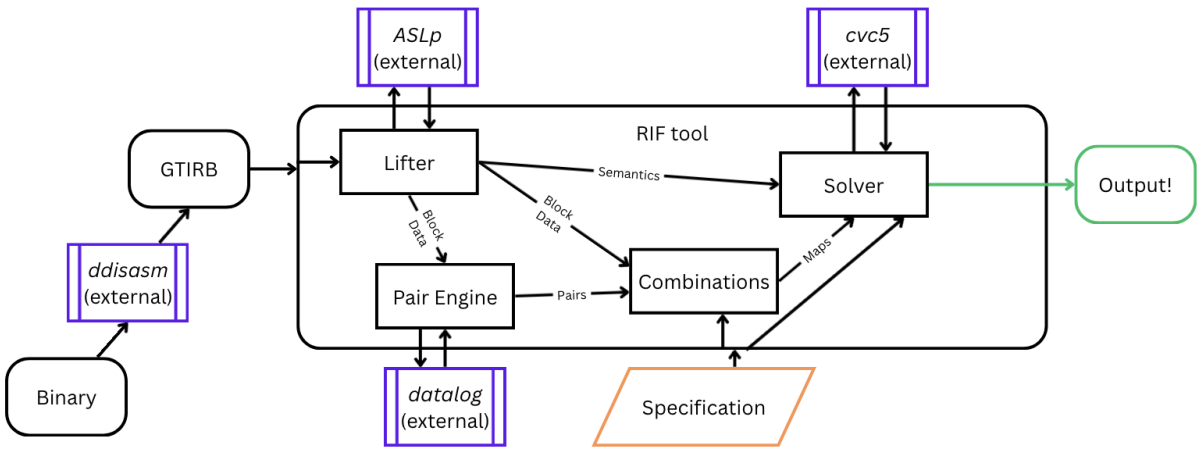

Figure 3.1: The Oregano architecture and submodules

```c
#include <atomic.h>

int y = 0;
int *x;

int foo(int b) {
  int a = *x;
  atomic_store_explicit(&y, 100, memory_order_relaxed);
  atomic_store_explicit(x, b, memory_order_relaxed);
  return a;
}
```

Figure 3.2: Original C code for *minimal.c* reordering example.

We discovered early in this process that an up-to-date ARMv8 compiler, `gcc` version 14, often reuses registers across simple C language statements, which removes all potential reordering behaviour from the code (as this introduces artificial register dependencies between the two stores). To resolve this, we resorted to the complex-looking snippet above, utilising the `atomic_store_explicit` functions, as well as other tricks with local variables. However, note that the compiler's choice to reuse or not reuse registers is totally opaque, and cannot be relied upon, similarly to the dead-code-elimination example in Section 1.3. These "atomic" functions also necessitate the inclusion of the `<atomic.h>` system header.

3.1 Compilation

From this example, our `foo()` function compiles down to eleven assembly instructions, presented in Fig. 3.3. At this level, the two "core" store instructions to `y` and `x` occur at lines 7 and 9, respectively, however, there are multiple other memory operations that could be reordered. This example demonstrates all of the contrivances we introduced earlier with *Secret Write*: `x` is stored to at instruction 9 using the address `[x2]`, which is extremely difficult to practically resolve – even taking into account the "symbol table", a global mapping from C variables to memory addresses, the address used by instruction 9 is loaded in instruction 8 based on a value originally provided in instruction 0, making it simply impossible to resolve this data based on only one pair of instructions.

Hopefully this demonstrates the choice of our "combinations" approach (Section 2.3.1) to searching the state space, rather than attempting to devise further static code analyses to resolve this information. However, further discussion of the possibility of static analysis to improve this tool is included in Section 4.1, and should be considered as a useful pre-solver step.

```
0  adrp    x2, 20000 <__data_start>
1  mov     w1, w0
2  add     x3, x2, #0x18     // 24
3  mov     w4, #0x64         // 100
4  ldr     x0, [x2, #24]
5  add     x3, x3, #0x8      // 8
6  ldr     w0, [x0]
7  str     w4, [x3]
8  ldr     x2, [x2, #24]
9  str     w1, [x2]
10 ret
```

Figure 3.3: Compiled assembly code for *minimal.c* reordering example.

3.2 Lifting

In discussing the compiled assembly code above, we have skipped over a key step in the disassembly. 64 bit binary executables are packaged into a format known as *elf64* [28], and parsing of this format to translate the raw bytes back into executable code is a non-trivial task in and of itself. To make this more manageable, Oregano uses off-the-shelf tools developed by GrammaTech, a software security research company. Their tool, *ddisasm* [29], takes the raw bytes of an *elf64* file and parses this into a data structure, which is serialised into the GrammaTech Intermediate Representation for Binaries (GTIRB) [30] format. GTIRB can be readily parsed by our tool, and this saves us writing our own parser for *elf64*. The output we receive is a high-level abstract syntax tree of "basic blocks", where each block points to other blocks, but does not have any internal syntax; the block's internal data is just a series of bytes.

The first important step that Oregano takes is to resolve data from the "symbol table" in order to locate the particular function that a user wants to RIF-check. Once this is done, Oregano uses *GTIRB*'s basic-block control flow graph and constructs a subset of the control flow graph containing exclusively that function's basic blocks. Some overapproximation occurs at this stage – it is possible that blocks are included that do not necessarily belong to the function under investigation – but this is a necessary evil that we shall leave to future work. Subsequently, we can extract the raw bytes of the executable that correspond to our desired function.

In order to turn these bytes into meaningful instruction-level data, Oregano then uses ASLp [31]. ASLp is a partial evaluator and lifter for ARMv8's Architecture Specification Language. In the context of Oregano ASLp provides the ability to translate the raw bytes of the executable code into a higher-level intermediate language with a recognisable – and traversable – abstract syntax tree structure.

ASLp lifts a sequence of 4 bytes, representing one assembly instruction, into a series of ASL statements. Each series represents the complete behaviour of that instruction, minus some complexities that we avoid

considering[1]. We can therefore use ASLp to turn our sequence of bytes into a sequence of miniature ASL programs, each one a short list of ASL statements detailing the complete behaviour of their corresponding instruction. In this way we end up with the full ASL behaviour for our desired function.

In our experience with Oregano, the most-commonly-reorderable memory instructions are variations on the basic `ldr` and `str` operations. Instructions 7 and 9 from our example are lifted left-to-right into the following (simplified) fragments of ASL:

```
0 str  w4, x3;
1 str  w1, x2;
```

```
0 Call('Mem.set', _R[4], _R[3])
1 Call('Mem.set', _R[1], _R[2])
```

Figure 3.4: ASL fragments for instructions 7 and 9 of *minimal.c* reordering example

To proceed to the next steps, Oregano collapses this extensive sequence of ASL programs down into a sequence of summaries. An example pair of summaries for instructions 7 and 9 are provided in Fig. 3.5. This process involves finding a minimal set of reordering-relevant facts about each instruction:

- All of the registers used by the instruction, categorised as loads, stores, reads, or writes, and
- Whether the instruction is a "control" instruction (e.g. a branch or a jump).

Furthermore, we also determine an additional series of facts relating to the original execution order of instructions. These facts represent the placement of individual instructions inside basic blocks, as well as the control-flow-graph's topological ordering of basic blocks. In addition to these summaries and ordering-related facts, Oregano still retains the full semantic data, as this is needed again later for the solver encoding.

The end result of our lifting phase is a mapping of "block data": each basic block from our function as part of a control-flow graph for that function, but also including the register summaries from Fig. 3.5 and the full ASL mini-programs from Fig. 3.4.

3.3 Pair Generation

The pair-generation portion of Oregano relies on Datalog, a logic programming language. Off-the-shelf libraries are again, as with the lifting step, available to link Datalog with the Oregano tool.

The implementation of ARMv8 reordering rules as Datalog relations is straightforward. Working backwards, reorderable pairs are defined as instruction pairs usually executed in order `A; B`, where both `A` and `B` are memory operations, and the ordering `A; B` is not preserved by the ARMv8 reordering rules. Order-preservation can occur by as a result of many conditions:

[1]For example, an ARMv8 processor has four distinct stack pointers based on the current execution context: operating system vs hypervisor vs user-land code and one unspecified [32]. Rather than generate code to manage all four, ASLp assumes code is running at a user-land level and does not require additional stack pointers.

Instruction 7: `str x4, [x3]`

```
{
  read = [Register 3; Register 4],
  write = [],
  load = [],
  store = [Register 3],
  control = false
}
```

Instruction 9: `str x1, [x2]`

```
{
  read = [Register 1; Register 2],
  write = [],
  load = [],
  store = [Register 2],
  control = false
}
```

Figure 3.5: Semantic summary for instructions 7 and 9 of *minimal.c* reordering example

- Two instructions (A, B) loading a from an address-register R, cannot reorder if R is used as a store address in-between them
- Two instructions (A, B) cannot reorder if B stores a value that has any dependency on A (a "data dependency")
- Two instructions (A, B) cannot reorder if B stores at an address that has any dependency on A (an "address dependency")
- The rules for "data dependency" and "address dependency" above are transitive – if B depends on an inbetween-instruction M, and M depends on A, then B and A cannot reorder[2]
- Two instructions cannot be reordered if there is any fence between them
- A store instruction cannot be reordered before any control-flow-affecting instruction

If datalog is unable to infer that any of these reordering-blocking relationships are true for a given pair of instructions, then they can be reordered.

Continuing with the summary example provided in Fig. 3.5, the two instructions are both memory operations, and their registers used are wholly disjoint. Furthermore, the only instruction between them (see Fig. 3.3) is Instruction 8: `ldr x2, [x2, #24]`. The processor can potentially execute both instruction 8 and instruction 9 before instruction 7, and the Datalog engine will discover this quite easily. At the end of this phase, our tool now has not only the "block data" mapping discussed above but also a further-filtered list of only those instruction pairs that may exhibit reordering.

Importantly, the choice to rely on a system of Datalog rules means the Oregano tool is parameterised in terms of its memory model. If we wish to conduct a RIF check for an alternative hardware weak memory model, we only need to supply a new set of Datalog rules that formalise the possible reordering behaviours and present to us a set of reorderable pairs. The possibility to extend Oregano in this way is discussed more in Section 4.1.

[2]This point is implemented in Datalog as two separate rules, giving us 7 dependencies total.

3.4 Sequential Solver Encoding

We have now boiled down our "component" under analysis to a series of pairs of reorderable instructions. It remains to check these instructions, to determine whether their reordering may cause problems for the program, using a theorem prover. In Section 2.2.1, we discussed the RIF system for proving that reordering an instruction pair does not interfere with the behaviour of the program, and subsequently in Section 2.3 we re-formulated this approach into showing that all the behaviour exhibited by re-ordered code is also exhibited by the ordinary, non-reordered code. Intuitively, this means:

> *For all possible initial-, middle-, and end-states resulting from a correct in-order execution, there must exist another series of middle-states for an out-of-order execution such that the program still starts in the same set of initial-states and ends in the same set of end-states.*

In practice, we have to universally quantify a *lot* of variables, as mechanised solvers do not always come with the notion of a singular state mapping, but this is not a concern. What is a concern is the existential quantifier sitting in the middle of this statement – ideally, we want to have all of our quantification on the outside of this formula for the solver to explore. Fortunately, there is an approach to solving this problem, known as Skolemisation [33]. The approach here is that first, we can move the existential quantification to the front of the formula by introducing a function $\mathcal{F}$ from all of our universally quantified variables to an element that was existentially quantified, instead of quantifying only the new variables in the middle. Formally, this is the equivalence

$$\forall\, x \in T_1, \exists\, y \in T_2.\ P(x, y) \iff \exists\, f \in (T_1 \to T_2).\ \forall\, x, P(x, f(x)) \tag{3.1}$$

for some "types" T_1 and T_2. This can be extended for arbitrarily many universally-quantified variables.

With Skolemisation, our RIF behavioral-subset can now be expressed in the following form:

$$\begin{aligned} &\exists\, \mathcal{F} \in (State \times State \times State \to State).\ \forall\, \sigma, \sigma', \sigma'', \\ &\qquad \sigma' = \beta_{\langle\alpha\rangle}(\mathcal{R}(\sigma)) \wedge \sigma'' = \alpha(\mathcal{R}(\sigma')) \implies \\ &\mathcal{F}(\sigma, \sigma', \sigma'') = \alpha(\mathcal{R}(\sigma)) \wedge \sigma'' = \beta(\mathcal{R}(\mathcal{F}(\sigma, \sigma', \sigma''))) \wedge \mathcal{G}(\sigma') \end{aligned} \tag{3.2}$$

In essence, here, we are treating $\mathcal{R}$, $\mathcal{G}$ and the instructions as functions over states, and we are asking the solver to show that there exists (a function returning) a middle-state for in-order execution so that the beginning and end states still line up between both executions. This is the standard way to argue about subsets: for all out-of-order executions, there exists an in-order execution. As a result of this process, our formulisation becomes a specialised form of function synthesis problem called "Exists/Forall" [34]. Rather than our original technique requiring us to prove the existence of a middle-state, we simply synthesise a function to return a middle-state satisfying our constraints. If we can perform this synthesis, then we have a proof.

For this, we can use a technique known as Syntax-Guided Synthesis (SyGuS) [35]. Our choice of solver is CVC5 [36]; this is motivated by its support for the SyGuS system as well as off-the-shelf bindings that we can use for linking into the Oregano pipeline. Fig. 3.6 illustrates (left-to-right) the "constraint flow" of

how each state is linked to the others by the instructions and relies, subsequently feeding into our desired synthesis function. In this figure, the maroon lines represent the assumed transitions corresponding to the out-of-order execution, while the green arrows show how these beginning and end states constrain our synthesised function.

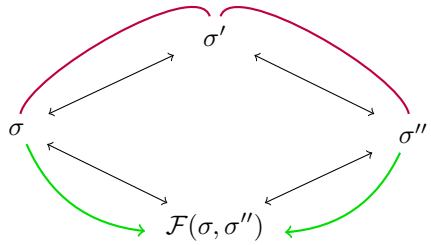

Figure 3.6: "Constraint flow" over program state transitions for a RIF-check with no specification.

An example of the system of constraints for our instructions 7 and 9 is shown in Fig. 3.7. At this step, the key detail is that our states must be split up into many individual variables; as we alluded to earlier, CVC5 does not support construction or use of an abstract state mapping. It is simple for the Oregano code to identify ahead of time and define all variables that will be needed for our check, though it does expand the work needed to automatically generate the constraints greatly.

For brevity, the arguments to the synthesis functions are omitted, but each function takes all twelve other state variables as input. Note our abstract state mapping is represented as a set of register or "memory-at-register" variables `R1`, `MR2`, `MR3`, `R4`, as well as prime & double-prime variants. We also have synthesis functions `fR4`, `fMR3`, ... for the reordered state. Due to the complexities involved in bitvector reasoning, we also simplify our state to be comprised of integers. In our experience, typical ARMv8 assembly does not combine complex arithmetic (where bitvector overflow may be relevant) with memory accesses in single instructions, so this choice seems reasonably safe.

Note that we are not directly asking it to show any implication in this stage, as it becomes implicit in the structure of our CVC5 query: we *assume* the LHS (that the state transitions from σ to σ' to σ'' occur), and then must *prove* the RHS (that we can develop a function satisfying our constraints as the middle-state for in-order execution). Nowhere do we tell the solver that some portion implies the other portion. This enables us to dispatch many small constraints and assumptions to the solver, increasing its effectiveness compared to less, larger constraints.

In Fig. 3.7, the top-left block encodes that over instruction 7, `R4` does not change, `R1` does not change, the memory-at-`R2` does not change, but the memory-at-`R3` is updated with the value of `R4`. The solution to the constraints that we have placed via instructions 7 and 9 is relatively simple: the function `fR4` returns `R4`, the function `fR1` returns `R1`, and the functions `fMR3` and `fMR2` return `MR3` and `R1`, respectively.

```
(synth-fun fR4 (...) Int)   ; Declare SyGuS functions, one-per-relevant-variable
(synth-fun fR1 (...) Int)
(synth-fun fMR3 (...) Int)
(synth-fun fMR2 (...) Int)

(declare-var R4 Int)        ; Declare registers ("for all")
(declare-var R1 Int)
(declare-var MR3 Int)
(declare-var MR2 Int)
...                         ; repeat for prime and double-prime
```

```
9⟨7⟩ {
(assume (= (R4 R4')))
(assume (= (R1 R1')))
(assume (= (MR3 MR3')))
(assume (= (R1 MR2')))

7 {
(constraint (= (R4 (fR4 (...)))))
(constraint (= (R1 (fR1 (...)))))
(constraint (= (R4 (fMR3 (...)))))
(constraint (= (MR2 (fMR2 (...)))))

7 {
(assume (= (R4' R4'')))
(assume (= (R1' R1'')))
(assume (= (R4' MR3'')))
(assume (= (MR2' MR2'')))

9 {
(constraint (= ((fR4 (...)) R4'')))
(constraint (= ((fR1 (...)) R1'')))
(constraint (= ((fMR3 (...)) MR3'')))
(constraint (= ((fR1 (...)) MR2'')))
```

Figure 3.7: Partial SyGuS-language solver code for instructions 7 and 9 of *minimal.c*

3.4.1 Symbolic Memory

An additional feature evident in Fig. 3.7 is the choice to collapse memory to single symbols `MRX`, representing the "memory-at-register-X". An alternative encoding could be to reason about `R3` and `R2` as integers used to index a large array of memory. We decided that this was a sub-optimal approach due to the difficulties solvers face when reasoning about arrays, and given our tool already uses our combinations-based approach from Section 2.3.1 to resolve aliasing behaviours, there is no real benefit to array-based memory representations.

Unfortunately, this does create some complexity when resolving non-immediate memory accesses – for example instruction 8 of Fig. 3.3 is `ldr x2, [x2, #24]`, a load from `x2` at an offset of 24 bytes. In this case, we have to take special care to create a memory variable for "memory-at-x2-plus-24", which is done by collapsing the entire ASL expression given as the address for the dereferencing operation.

Fortunately however this does not affect our combination-generation approach to resolving aliasing. We don't concern ourselves with the value of `x2`, only the relevant information: whether "memory-at-x2-plus-24" is a memory alias for any of our specified variables.

3.5 Combinations & Concurrent Solver Encoding

As of now, we have explained how our solver can prove that reordering instructions exhibits the same behaviours within a single thread. This is non-trivial in itself, but ultimately, it is the same guarantee that the processor already provides to us when it performs reordering to begin with.

In order to demonstrate that the pair is safe in a concurrent context, we must now take in our specification, as described in Section 2.3.1. We can then turn the rely condition into uninterpreted functions over variables, as performed in [6], and generate further states that demonstrate the effects of the environment on the execution.

Our constraint flow with added rely conditions looks like this:

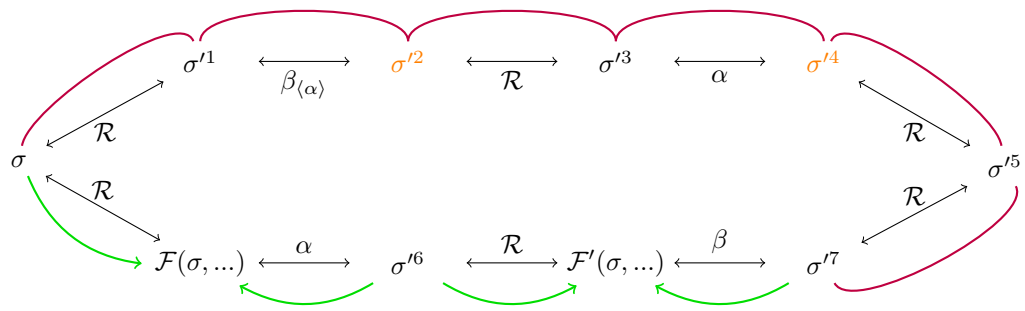

Figure 3.8: "Constraint flow" over program state transitions for a RIF-check with specification.

In this diagram note that the overlaid maroon trail denotes *assumed* state transitions, while the directional green arrows denote specific constraints on our state-generating SyGuS functions $\mathcal{F}$ and $\mathcal{F}'$. We also apply the guarantee condition as a constraint on σ'^2 and σ'^4, highlighted in orange, ensuring that the re-ordered instructions still uphold that portion of the specification.

The reason why we rely so heavily on CVC5 assumptions for state transitions is due to a difficulty in the use of CVC5 variables. When we declare the register-variables for each state, we are implicitly quantifying over all possible values that they may take. If, then, we *constrain* that some variable `R0` is equal to `R0'` – say, because we are transitioning over an assembly-language instruction that does not modify its value – then CVC5 will invariably fail to produce a result. This is because of the implicit quantification: if CVC5 chooses that `R0` holds the value 4 and `R0'` holds the value 5, which it is within its rights to do, then we are "constraining" the system to find functions such that $5 = 4$, an impossible requirement. Instead, we utilise assumptions for all state transitions that do not involve the SyGuS functions $\mathcal{F}$ and $\mathcal{F}'$, as any constraints will prove to never be satisfiable.

Having descibed this full sequentialised system of constraints, we must perform the two checks described in Section 2.3.1. This results in a call to CVC5 to check the in-order code, and if that succeeds, a second call to check the full out-of-order system shown in Fig. 3.8. Developing tool support to generate all transitions from this diagram now becomes an engineering effort, with many cases and pitfalls to consider.

To list these requirements out:

- We must ensure that we generate constraints for our aliasing assumptions (i.e. those that link our memory variables `MR0` with our program variables `X`) in *all* states. Registers used as memory addresses should never change values[3], as this would break the CPU-based independence guarantee between instructions, meaning this assumption is safe, and necessary to link up our program-variable-modifying rely with our memory-variable-modifying instructions.
- We must ensure that we generate initial constraints that align with our other program-state-exploration "combinations". The uninterpreted functions that comprise these combinations must be appropriately defined and typed.
- We need some way of translating the AST of the specification (relies and guarantees) into CVC5 terms, which can be applied as constraints or assumptions. When we do this, we must ensure that all un-modified items of the state have terms linking them to their prior values.
- We need some way of translating the ASL syntax of our instructions into CVC5 terms, much the same as our specification above. Again, un-modified items from the state must be linked to their prior values.

With these requirements, the tool must generate the first CV5 problem – linking together four states via $\mathcal{R}, \alpha, \mathcal{R}, \beta$ and constraining states 2 and 4 using $\mathcal{G}$ – then if this succeeds, the much more complex reordering problem: linking together seven states (plus two function-states) via $\mathcal{R}, \alpha, \mathcal{R}, \beta, \mathcal{R}$ and $\mathcal{R}, \beta_{\langle\alpha\rangle}, \mathcal{R}, \alpha, \mathcal{R}$ and applying $\mathcal{G}$ to the two out-of-order post-instruction states.

3.6 Evaluation

Oregano is fully capable of demonstrating that reordering maintains sequential guarantees. Unfortunately, the concept for the program-state-exploring system was only fully developed late in the available time, leaving not much room for implementation.

Skeleton work has begun to complete the encoding of concurrent behaviour into CVC5, including generating all aliasing combinations and utilising them to link program and memory state. Starting with a trivial rely/guarantee specification – the state changes nothing, and this thread can do anything – scaffolding is also there to encode the specification as further state transitions. At this stage, there remains issues to be solved with program variables remaining unlinked over state transitions, leading to the unsolvable-constraint problem described earlier: for all `R4'6`, `R4'7`, we cannot generate a function `fR4(...)` such that `fR4(...) = R4'6` *and* `fR4(...) = R4'7` unless the remainder of our assumptions suffice to show that `R4'6 = R4'7` anyway.

Nevertheless, the success of the program in what was successfully implemented is promising for future development. Over five small examples – *minimal.c* as shown in Fig. 3.2 but also a version of *Secret Write* as shown in Fig. 1.1 (with 8 instructions for each thread) and a small spin-lock example (with

[3]This has been observed in all instruction pairs we saw.

2 instructions for unlock and 5 for lock) – the binary lifting and pair generation consistently finished under 2.3s. On the examples from *shuffle.c*, which are far more contrived examples simply designed to maximise the number of assembly instructions that could reorder, the pair-generation still scales somewhat favourably. Unfortunately, at some point this scaling breaks down, likely due to the increases in "fact" numbers that Datalog must manage. The `execution_order` relation grows quadratically in the number of instructions (as it generates the true order-of-execution for all pairs of instructions), which quickly reaches 82621 ordering relations (let alone the other facts) for the last *shufffle.c* example in Fig. 3.9. All tests in this table were conducted on a PC with an AMD Ryzen 9 7950X CPU; reported times are the mean of 5 runs.

Example		Instructions	Time
minimal.c	foo()	11	2.265s ±0.022s
spinlock.c	lock()	5	2.243s ±0.004s
	unlock()	2	2.227s ±0.004s
secret_write.c	reader()	8	2.246s ±0.001s
	writer()	8	2.238s ±0.004s
shuffle.c	shuffle1()	113	5.333s ±0.008s
	shuffle2()	211	23.51s ±0.018s
	shuffle3()	407	176.26s ±0.19s

Figure 3.9: Timing results for lifting + pair-generation for various functions

When extending these generated pairs by performing the sequential RIF check from Section 3.4, *minimal.c* only takes another 0.275-0.325s to dispatch the constraints to the solver and receive a result. This is promising for our combinatorial approach to the state space search: if this problem can be dispatched in 0.3s then we anticipate being able to substantially reduce the state-space in fairly short time.

Chapter 4

Conclusion

We have now seen the full skeleton of Oregano, which we propose as a tool to automate RIF checks for ARMv8 binary code. Though it is not complete, it serves as a useful proof-of-concept and demonstrates the practicability of building such a tool. Oregano also enables us to identify several early pitfalls, including that pair generation is likely to be the tool's first bottleneck.

4.1 Future Work

There are several avenues available to continue this work; filling out the remainder of the work required to perform RIF checks with full concurrency is the most obviously pressing. To that end, future work would ideally begin by implementing a specification parser and resolving the outstanding issues with contradictory constraints being fed to the solver. If those tasks are completed to reach a fully-functional RIF tool, there does remain plenty of work for further extensions.

Use of improved static analyses as a further filtering step could greatly reduce the number of combinations to be tested. For instance, a copy-propagation pass and points-to analysis have the potential to totally resolve the aliasing behaviour of reordered pairs, or at least, reduce the state space that the state-partitioning combinations must cover.

As mentioned towards the end of Section 3.6, the tool does begin to break down for larger and larger functions, due to the increased volume of facts provided for datalog to infer from. While the implementation of Oregano relies on an external Datalog implementation, further work could optimise it, optimise its usage, or even consider a customised inference engine specialised for the needs of reorderability-analysis for assembly code. Another trick we have up our sleeves is SyGuS's grammar functionality: improving solver performance by constraining the grammar of the function it is asked to generate [35]. This Could be investigated if the solver begins to break down for larger examples.

Also as mentioned during the pair generation in Section 3.3 is the possibility to extend the tool to consider alternative weak memory models. This could be achieved by simply swapping out the Datalog rules under consideration for that of a different memory model. Even with the deep dependency on ASLp, limiting Oregano to only consider the ARMv8 instruction set, there are several other weak memory models that may arise on an ARMv8 CPU when managing memory-mapped hardware devices

[18], [32]. Tool support to investigate these models' reordering behaviour would be valuable.

Finally, it would be remiss not to suggest that future work consider the practical use of this tool to investigate the possibility of reordering-based bugs on larger and more realstic, practical systems. A motivating example for the proposal of this work was OpenBSD's `ice(4)` driver [37], a piece of software that has been noted by it's authors to manage complex instruction-level memory-ordering behaviour.

4.2 Final Remarks

Reasoning about programs, especially security properties of concurrent programs, is difficult. Modern hardware compounds this difficulty by introducing weak memory behaviour – the notion that programs no longer necessarily execute instructions in the order they are written.

Existing approaches generally focus on model checking over compositional reasoning. While this is highly automatable, the lack of programmer input is both a blessing and a curse, as without programmer-provided summaries of thread or function behaviour, the reasoning tool must investigate all possible paths, and even with advanced reduction techniques, may still run out of time or memory.

The RIF system [14] avoids this by providing weak memory checks as a supplement to existing compositional reasoning techniques. By computing all pairs of instructions that may execute out-of-order, and checking these pairs against the concurrent specification of the program independent of the context in which they actually execute, weak memory behaviour can be discounted, and an ordinary proof of correctness under sequential consistency is enough.

We devised the backbone of a tool for automating the process of RIF checks, and wrote a prototype tool following this design. The tool unfortunately does not hit all of our objectives to be considered a complete tool for RIF checks, but is able to establish the correctness of sequential reordering. Future work is described to extend this prototype tool with the remainder of the steps needed to establish full, concurrent RIF, as well as several extensions to improve automation of the contemporary RIF approach.

Bibliography

[1] L. Lamport, "Proving the correctness of multiprocess programs," *IEEE Trans. Softw. Eng.*, vol. 3, no. 2, pp. 125–143, Mar. 1977, ISSN: 0098-5589. DOI: 10.1109/TSE.1977.229904. [Online]. Available: https://doi.org/10.1109/TSE.1977.229904.

[2] A. Sabelfeld and A. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003. DOI: 10.1109/JSAC.2002.806121.

[3] K. Winter, N. Coughlin, and G. Smith, "Backwards-directed information flow analysis for concurrent programs," in *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*, 2021, pp. 1–16. DOI: 10.1109/CSF51468.2021.00017.

[4] N. Coughlin and G. Smith, "Rely/Guarantee Reasoning for Noninterference in Non-Blocking Algorithms," in *2020 IEEE 33rd Computer Security Foundations Symposium (CSF)*, 2020, pp. 380–394. DOI: 10.1109/CSF49147.2020.000034.

[5] I. J. Hayes and C. B. Jones, "A Guide to Rely/Guarantee Thinking," in *Engineering Trustworthy Software Systems*, J. P. Bowen, Z. Liu, and Z. Zhang, Eds., Cham: Springer International Publishing, 2018, pp. 1–38, ISBN: 978-3-030-02928-9.

[6] G. Smith, "A Dafny-based approach to thread-local information flow analysis," in *2023 IEEE/ACM 11th International Conference on Formal Methods in Software Engineering (FormaliSE)*, 2023, pp. 86–96. DOI: 10.1109/FormaliSE58978.2023.00017.

[7] D. Peled, P. Pelliccione, and P. Spoletini, "Model Checking," in *Wiley Encyclopedia of Computer Science and Engineering*. John Wiley & Sons, Ltd, 2009, pp. 1904–1920, ISBN: 9780470050118. DOI: https://doi.org/10.1002/9780470050118.ecse247. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/9780470050118.ecse247. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/9780470050118.ecse247.

[8] P. Godefroid, "Model checking for programming languages using VeriSoft," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '97, Paris, France: Association for Computing Machinery, 1997, pp. 174–186, ISBN: 0897918533. DOI: 10.1145/263699.263717. [Online]. Available: https://doi.org/10.1145/263699.263717.

[9] M. Kokologiannakis and V. Vafeiadis, "HMC: Model Checking for Hardware Memory Models," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20, Lausanne, Switzerland: Association for Computing Machinery, 2020, pp. 1157–1171, ISBN: 9781450371025. DOI: 10.1145/3373376.3378480. [Online]. Available: https://doi.org/10.1145/3373376.3378480.

[10] P. A. Abdulla, S. Aronis, M. F. Atig, B. Jonsson, C. Leonardsson, and K. Sagonas, "Stateless model checking for TSO and PSO," *Acta Inf.*, vol. 54, no. 8, pp. 789–818, Dec. 2017, ISSN: 0001-5903. DOI: 10.1007/s00236-016-0275-0. [Online]. Available: https://doi.org/10.1007/s00236-016-0275-0.

[11] G. Balakrishnan and T. Reps, "WYSINWYX: What you see is not what you eXecute," *ACM Trans. Program. Lang. Syst.*, vol. 32, no. 6, Aug. 2010, ISSN: 0164-0925. DOI: 10.1145/1749608.1749612. [Online]. Available: https://doi.org/10.1145/1749608.1749612.

[12] M. Howard, "Some Bad News and Some Good News," 2006. [Online]. Available: https://learn.microsoft.com/en-us/previous-versions/ms972826(v=msdn.10).

[13] W. Chen, P. Chang, T. Conte, and W. Hwu, "The effect of code expanding optimizations on instruction cache design," *IEEE Transactions on Computers*, vol. 42, no. 9, pp. 1045–1057, 1993. DOI: 10.1109/12.241594.

[14] N. Coughlin, K. Winter, and G. Smith, "Rely/Guarantee Reasoning for Multicopy Atomic Weak Memory Models," in *Formal Methods*, M. Huisman, C. Păsăreanu, and N. Zhan, Eds., Cham: Springer International Publishing, 2021, pp. 292–310, ISBN: 978-3-030-90870-6.

[15] J. Alglave, L. Maranget, and M. Tautschnig, "Herding Cats: Modelling, Simulation, Testing, and Data Mining for Weak Memory," *ACM Trans. Program. Lang. Syst.*, vol. 36, no. 2, Jul. 2014, ISSN: 0164-0925. DOI: 10.1145/2627752. [Online]. Available: https://doi.org/10.1145/2627752.

[16] R. J. Colvin and G. Smith, "A Wide-Spectrum Language for Verification of Programs on Weak Memory Models," in *Formal Methods*, K. Havelund, J. Peleska, B. Roscoe, and E. de Vink, Eds., Cham: Springer International Publishing, 2018, pp. 240–257, ISBN: 978-3-319-95582-7.

[17] S. Flur et al., "Modelling the ARMv8 architecture, operationally: concurrency and ISA," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '16, St. Petersburg, FL, USA: Association for Computing Machinery, 2016, pp. 608–621, ISBN: 9781450335492. DOI: 10.1145/2837614.2837615. [Online]. Available: https://doi.org/10.1145/2837614.2837615.

[18] A. Limited, "Armv8-A memory model," 2019. [Online]. Available: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Learn%20the%20Architecture/Armv8-A%20memory%20model%20guide.pdf?revision=58b1dd0a-3800-4218-b21a-f95a0332034c.

[19] S. V. Adve and H.-J. Boehm, "Memory models: a case for rethinking parallel languages and hardware," *Commun. ACM*, vol. 53, no. 8, pp. 90–101, Aug. 2010, ISSN: 0001-0782. DOI: 10.1145/1787234.1787255. [Online]. Available: https://doi.org/10.1145/1787234.1787255.

[20] H. Akkary, S. Srinivasan, R. Koltur, Y. Patil, and W. Refaai, "Perceptron-Based Branch Confidence Estimation," in *10th International Symposium on High Performance Computer Architecture (HPCA'04)*, 2004, pp. 265–265. DOI: 10.1109/HPCA.2004.10002.

[21] M. Kokologiannakis and V. Vafeiadis, "GenMC: A Model Checker for Weak Memory Models," in *Computer Aided Verification: 33rd International Conference, CAV 2021, Virtual Event, July 20–23, 2021, Proceedings, Part I*, Berlin, Heidelberg: Springer-Verlag, 2021, pp. 427–440, ISBN: 978-3-030-81684-1. DOI: 10.1007/978-3-030-81685-8_20. [Online]. Available: https://doi.org/10.1007/978-3-030-81685-8_20.

[22] T. L. Bui, K. Chatterjee, T. Gautam, A. Pavlogiannis, and V. Toman, "The reads-from equivalence for the TSO and PSO memory models," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Oct. 2021. DOI: 10.1145/3485541. [Online]. Available: https://doi.org/10.1145/3485541.

[23] M. Kokologiannakis, A. Raad, and V. Vafeiadis, "Effective lock handling in stateless model checking," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, Oct. 2019. DOI: 10.1145/3360599. [Online]. Available: https://doi.org/10.1145/3360599.

[24] E. M. Clarke, R. Enders, T. Filkorn, and S. Jha, "Exploiting symmetry in temporal logic model checking," *Formal Methods in System Design*, vol. 9, no. 1, pp. 77–104, Aug. 1996, ISSN: 1572-8102. DOI: 10.1007/BF00625969. [Online]. Available: https://doi.org/10.1007/BF00625969.

[25] Rich Felker, et al., *musl libc*, 2020. [Online]. Available: https://musl.libc.org/.

[26] O. Lahav and V. Vafeiadis, "Owicki-Gries Reasoning for Weak Memory Models," in *Automata, Languages, and Programming*, M. M. Halldórsson, K. Iwama, N. Kobayashi, and B. Speckmann, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 311–323, ISBN: 978-3-662-47666-6.

[27] S. Owicki and D. Gries, "An axiomatic proof technique for parallel programs I," *Acta Informatica*, vol. 6, no. 4, pp. 319–340, 1976, Cited by: 705. DOI: 10.1007/BF00268134. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-0017173886&doi=10.1007%2fBF00268134&partnerID=40&md5=a2e7e721be4c2b02390b8ba5b214e169.

[28] T. I. S. (TIS), "Executable and Linking Format (ELF) Specification," version 1.2, 1995. [Online]. Available: https://refspecs.linuxbase.org/elf/elf.pdf.

[29] A. Flores-Montoya and E. Schulte, "Datalog Disassembly," in *29th USENIX Security Symposium (USENIX Security 20)*, USENIX Association, Aug. 2020, pp. 1075–1092, ISBN: 978-1-939133-17-5. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/flores-montoya.

[30] E. Schulte, J. Dorn, A. Flores-Montoya, A. Ballman, and T. Johnson, *GTIRB: Intermediate Representation for Binaries*, 2020. arXiv: 1907.02859 [cs.PL].

[31] K. Lam and N. Coughlin, "Lift-off: Trustworthy ARMv8 semantics from formal specifications," in *2023 Formal Methods in Computer-Aided Design (FMCAD)*, 2023, pp. 274–283. DOI: 10.34727/2023/isbn.978-3-85448-060-0_36.

[32] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th. Wiley, 2012.

[33] T. Skolem, "Logisch-kombinatorische Untersuchungen über die Erfüllbarkeit oder Beweisbarkeit mathematischer Sätze nebst einem Theoreme über dichte Mengen," 1920.

[34] B. Dutertre, "Solving Exists/Forall Problems With Yices," in *13th International Workshop on Satisfiability Modulo Theories (SMT 2015)*, Jul. 2015.

[35] S. Padhi, E. Polgreen, M. Raghothaman, A. Reynolds, and A. Udupa, *The SyGuS Language Standard Version 2.1*, 2023. arXiv: 2312.06001 [cs.PL]. [Online]. Available: https://arxiv.org/abs/2312.06001.

[36] H. Barbosa et al., "cvc5: A Versatile and Industrial-Strength SMT Solver," in *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I*, D. Fisman and G. Rosu, Eds., ser. Lecture Notes in Computer Science, vol. 13243, Springer, 2022, pp. 415–442. DOI: 10.1007/978-3-030-99524-9_24. [Online]. Available: https://doi.org/10.1007/978-3-030-99524-9%5C_24.

[37] S. Sperling and D. Gwynne, *OpenBSD, sys/dev/pci/if_ice.c*, version 1.28, Computer Software, 2024. [Online]. Available: https://github.com/openbsd/src.